docker-dev — Specifica tecnica e formati

Versione: 0.1
Data: 13 ottobre 2026
Stato: bozza consolidata della baseline; non implementazione collaudata.

Le decisioni approvate derivano dalla discussione del progetto. Le raccomandazioni non approvate restano marcate APERTO. La CR originale e la revisione 2 sono documenti storici, non più riferimenti normativi. Il prototipo è una baseline comportamentale, non uno schema da riprodurre senza modifiche.

1. Componenti e layout

```
docker-dev/
  docker-dev.sh
  apply-templates.sh
  bin/docker-dev-runtime.pl          # nome indicativo
  bin/docker-dev-services.pl         # nome indicativo
  lib/DockerDev/                     # organizzazione interna da finalizzare
  langs/Python/
    runtime.yaml
    templates/{Dockerfile,entrypoint.Python.sh,.docker_container/...}
    compose/compose.yaml.tt           # posizione proposta, 008
    bin/{10.setup-volumes.sh,20.build.sh,30.services.sh,40.run.sh}
  services/<tipo>/
    service.yaml
    templates/compose.yaml.tt
    bin/                             # preparazione specifica
<progetto>/.docker-dev/
  runtime.yaml
  compose.user.yaml                  # supporto/precedenza 008
  generated/
  state/
```

apply-templates.sh consegna file e compone bashrc; non renderizza TT. Resolver e renderer sono responsabilità logiche, non obbligo di due programmi separati. La migrazione completa del main a Perl è successiva.

2. Pipeline

1. Parsing CLI rispettando --; scelta operazione prima della normale pipeline.
2. Risoluzione progetto/identità/contesto.
3. Lettura e composizione runtime, piano preliminare senza effetti Docker.
4. Applicazione file di progetto quando l'operazione lo richiede.
5. Conferma delle azioni mutanti secondo contratto da finalizzare.

6. 10.setup-volumes: risorse di linguaggio.
7. 20.build: immagine sviluppo se richiesta/mancante secondo O06.
8. 30.services: resolver generico, preparazione servizi e completamento Compose.
9. Risoluzione immagine, modello finale, fingerprint e label.
10. Validazione configurazione finale; esecuzione tramite 40.run.

L'ordine preciso tra validazione preliminare, conferma e preparazione deve distinguere controlli puri da quelli possibili soltanto dopo build/pull. La generazione completa con fingerprint richiede l'immagine risolta.

3. runtime.yaml — schema minimo

Stato: struttura consolidata; tipi/default sotto indicati sono proposta contrattuale da finalizzare in M1 (O12).

```
schema_version: 1
environment:
  APP_MODE: development
env_files:
  - ./development.env
ports: []
mounts:
  - type: bind
    source: ./datasets
    target: /datasets
    read_only: true
extra_hosts:
  host.docker.internal: host-gateway
shm_size: 1g
resources:
  memory: 2g
  cpus: 2
```

Campi supportati v1: environment, env_files, ports, mounts, extra_hosts, shm_size, resources. Nessun lifecycle, hook, strategia ownership o sezione volumes nel formato runtime comune.

- Mapping root e schema_version richiesti; chiavi sconosciute → errore.
- Environment: mapping con valori stringa; precedenza esplicita sui file ambiente.
- Env_files: lista; percorsi progetto relativi a PROJECT_DIR; base linguaggio da definire O12.
- Ports: target, published, host_ip, protocol; niente pubblicazione implicita. Default host_ip e tipi definitivi O12.
- Mounts: type bind/volume, source, target, read_only. Target assoluto; source bind assoluto dopo risoluzione; source volume nome Docker effettivo.
- Resources: solo campi supportati e validati, non passthrough arbitrario.

Standard Python

```
schema_version: 1
mounts:
  - type: bind
    source: "${context.project_dir}"
    target: /app
  - type: volume
    source: "${context.volume_name}"
    target: /app/.venv
  - type: volume
    source: uv-python
    target: "/home/${context.host_user}/.local/share/uv/python"
```

I riferimenti sono stringhe YAML risolte dal tool, non espressioni shell o Compose. Allowlist proposta: project_dir, volume_name, host_user, project_id. Nessuna espansione ambiente generalizzata. Confinare la risoluzione ai campi previsti; i $ letterali del payload devono essere trattati separatamente.

Composizione

- environment ed extra_hosts: progetto prevale per chiave.
- env_files: linguaggio seguito da progetto, precedenza documentata.
- mounts: concatenazione controllata, target duplicato → errore; mount annidati ammessi quando previsti.
- ports: concatenazione con controllo conflitti; controlli statici non garantiscono disponibilità runtime.
- shm_size/resources: override semplice sui campi supportati.

Non emulare merge Compose generale. Modifiche a queste regole richiedono decisione e test.

4. Volumi e mount

10.setup-volumes prepara venv e uv-python con nomi del contesto. Il runtime dichiara uso/mount, non creazione. Volumi aggiuntivi dev devono esistere dopo setup: assenza → errore. Lifecycle avanzato è fuori v1.

Renderer deduplica nomi Docker e genera chiavi interne per i volumi:

```
volumes:
  runtime-volume-1:
    external: true
    name: venv.dev.python.foo
```

Nessun chown automatico dei volumi servizi con UID host. Setup specifico nei relativi hook. Volumi dei servizi possono seguire politiche differenti, da specificare nel manifest (O13).

5. Catalogo servizi

Alias dichiarativi globali e/o progetto; sede e precedenza O04. Resolver legge manifest, parametri ammessi, template e hook senza case specifici per servizio.

Schema service.yaml ancora da definire: versione, immagine locale/upstream, parametri tipizzati, profili opzionali, volumi, healthcheck, integrazione dev, hook confinati al catalogo. Porta e connection_env non obbligatori per ogni servizio.

TT renderizza frammenti YAML, poi parser e aggregatore costruiscono un unico documento. Rifiutare duplicate keys e conflitti. Non concatenare documenti con services ripetuto. Serializzare scalari/liste/booleani; STRICT richiede flag sempre definiti. Template/hook sono codice fidato, parser sicuro non li rende sandbox.

6. Contesto e piano

Main comunica valori espliciti, non variabili Bash non esportate. JSON è proposta preferita (O14), con schema_version, operation, project, runtime, host, mounts, services e azioni risolte. Argomenti comando come lista, mai eval.

Piano preliminare: richieste e azioni. Piano applicabile: nomi/percorsi risolti, immagine disponibile, servizi e runtime finale. Distinguere dati pubblicabili e segreti.

7. Esecuzione Compose

Project name distinto dal container name; punti non ammessi nel primo. Politica ID/naming O01.

Effimero: compose run --rm --name <nome> --service-ports dev [argv]. Nessun comando artificiale sleep infinity; entrypoint immagine preservato. Modalità batch senza TTY secondo CLI.

Persistente: container normale del servizio Compose è proposta raccomandata; meccanica create/start/attach definitiva O05. Non usare run come comando di riavvio. Container attivo: attach/exec espliciti.

Servizi: up soltanto nomi selezionati; skip se lista vuota. Garantire readiness (O07). Non up generale seguito da run dev. Task con container persistente: nome/lifecycle separato O05.

8. Fingerprint

Approvato: SHA-256 di JSON deterministico della creation_spec normalizzata. JSON::PP canonical+utf8 e Digest::SHA sha256_hex. Non hash del YAML o del filesystem.

Includere versione fingerprint, ID immagine dopo build/pull, mount, ambiente finale letto da env_files, porte, rete rilevante, hostname, override utente/workdir/entrypoint/comando, TTY/stdin, risorse e sicurezza supportate.

Escludere timestamp, stato, sorgenti, contenuto venv, fingerprint stesso. Mapping canonici; liste ordinate solo se ordine semanticamente irrilevante. Distinguere comando assente da lista

esplicita. Fingerprint computato dopo tutti gli override: nessuna modifica successiva ai campi coperti.

```
use JSON::PP ();
use Digest::SHA qw(sha256_hex);
my $json = JSON::PP->new->canonical(1)->utf8(1)->encode($creation_spec);
my $fingerprint = 'sha256:' . sha256_hex($json);
```

L'algoritmo di normalizzazione deve essere specificato/testato; JSON canonico non normalizza automaticamente i dati. Hash non prova autenticità e può rivelare uguaglianza dei valori; non esporre segreti o snapshot non protetti (O10).

9. Label e inspect

Namespace illustrativo, da fissare O15:

```
labels:
  dev.docker-dev.managed: "true"
  dev.docker-dev.project-id: "<id>"
  dev.docker-dev.role: "dev"
  dev.docker-dev.fingerprint-version: "1"
  dev.docker-dev.runtime-hash: "sha256:..."
```

Codice comune definisce e genera label in services.dev.labels, non Dockerfile o valori utente. Lifecycle registrato separatamente o label dedicata: contratto O02/O15.

Inspect legge Config.Labels, State e Image. Controlli obbligatori: identità, versione/hash, stato; confronto ID immagine come verifica economica/coerenza. Non ricostruire tutti i mount per il normale riutilizzo. Inspect assente distinto da errore daemon/permessi.

Fermo compatibile → start; attivo compatibile → operazione esplicita; estraneo → errore; incompatibile → ricreazione autorizzata. Vecchi container senza label non adottati automaticamente. Modifiche filesystem interno fuori dalla compatibilità di creazione.

10. Stato, sicurezza e baseline

Snapshot applicato necessario per down senza --with e diff configurazione. Hash decide uguaglianza, snapshot spiega differenze. Permessi, retention, locking e redazione segreti O09–O11.

Baseline da verificare separatamente: ownership solo prima creazione; init fallita su volume esistente; uv-python condiviso; build TZ non consumato come ARG; immagine condivisa tra UID diversi; uv sync policy; splitting whitespace vecchi extra args.

Materiali mancanti per collaudo: .settings/.bashrc, .settings/.gitignore, configurazioni Python locali, servizi reali, versioni dipendenze e ambiente Docker/Compose.